

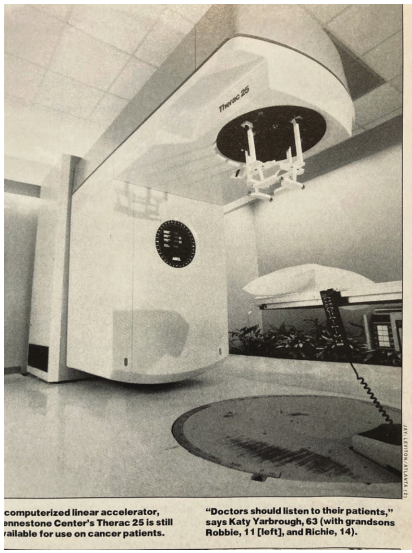




Abb.: https://testomat.io/blog/software-bug-cost/

Wie kann man formal korrekte SW schreiben

Sepp Stage

TU Chemnitz, Fakultät Betriebssysteme



TECHNISCHE UNIVERSITÄT
CHEMNITZ

Wie kann man formal korrekte SW schreiben*

*unter bestimmten Annahmen

Sepp Stage

TU Chemnitz, Fakultät Betriebssysteme



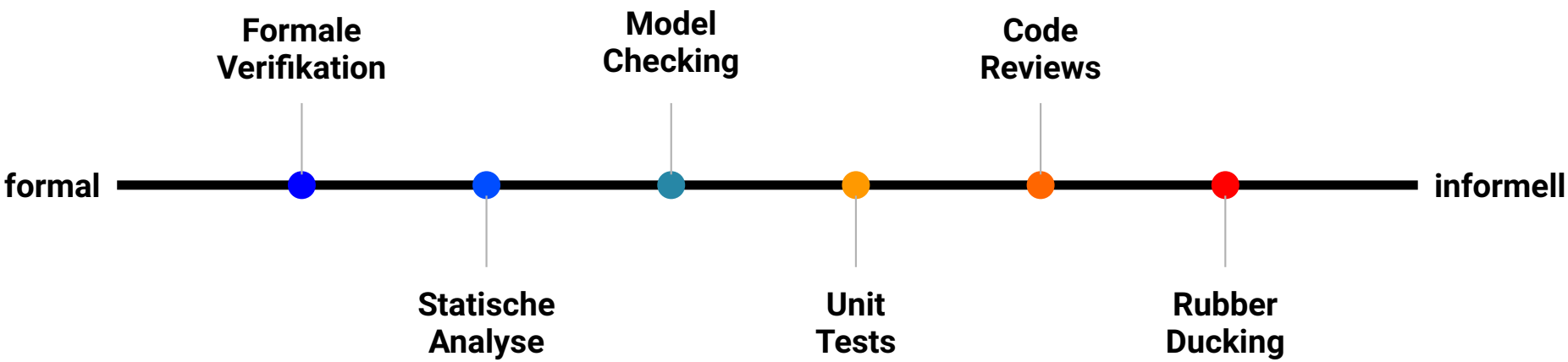
TECHNISCHE UNIVERSITÄT
CHEMNITZ

1. Methoden zur Sicherstellung von Codequalität

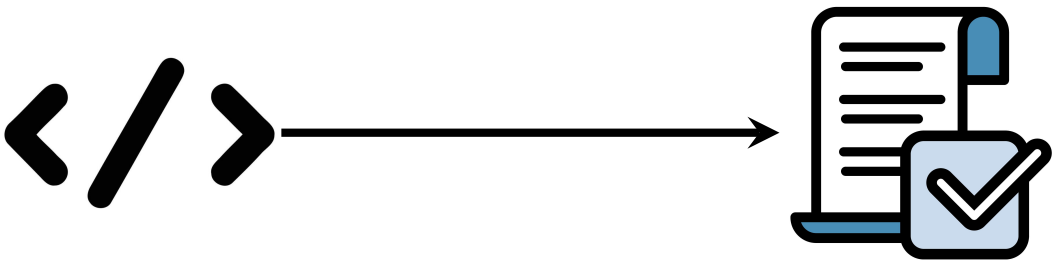
2. Verbindung Formalisierung - Implementation

3. Forschungsziel
   - SQuIRRL-Stack
   - Zu beweisende Eigenschaft
   - Isabelle Ansatz
   - Zusammenfassung

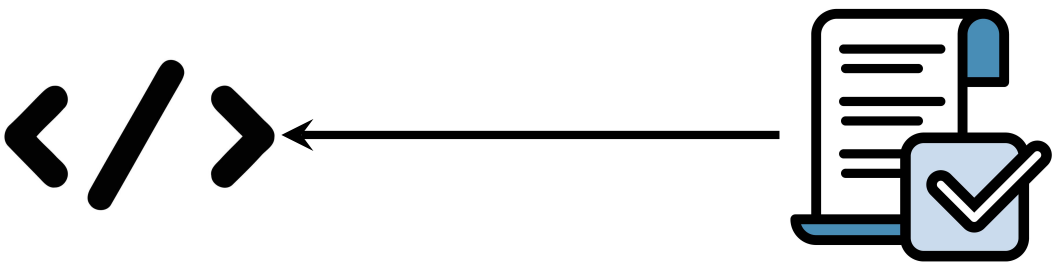
Methoden zur Sicherstellung von Codequalität



Verbindung Formalisierung - Implementation



+ Effizienter Code
− Abstraktion komplex



+ Änderung des Models generiert neuen Code
− Codegenerierung schwer steuerbar

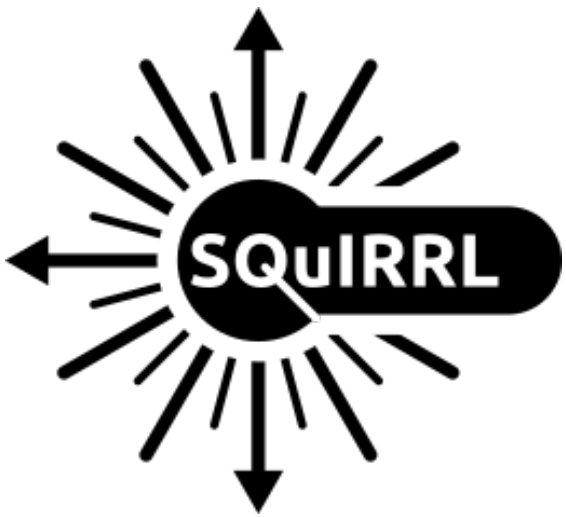


Figure 4-1 Cybercrime Trends in the US: Last 12 years

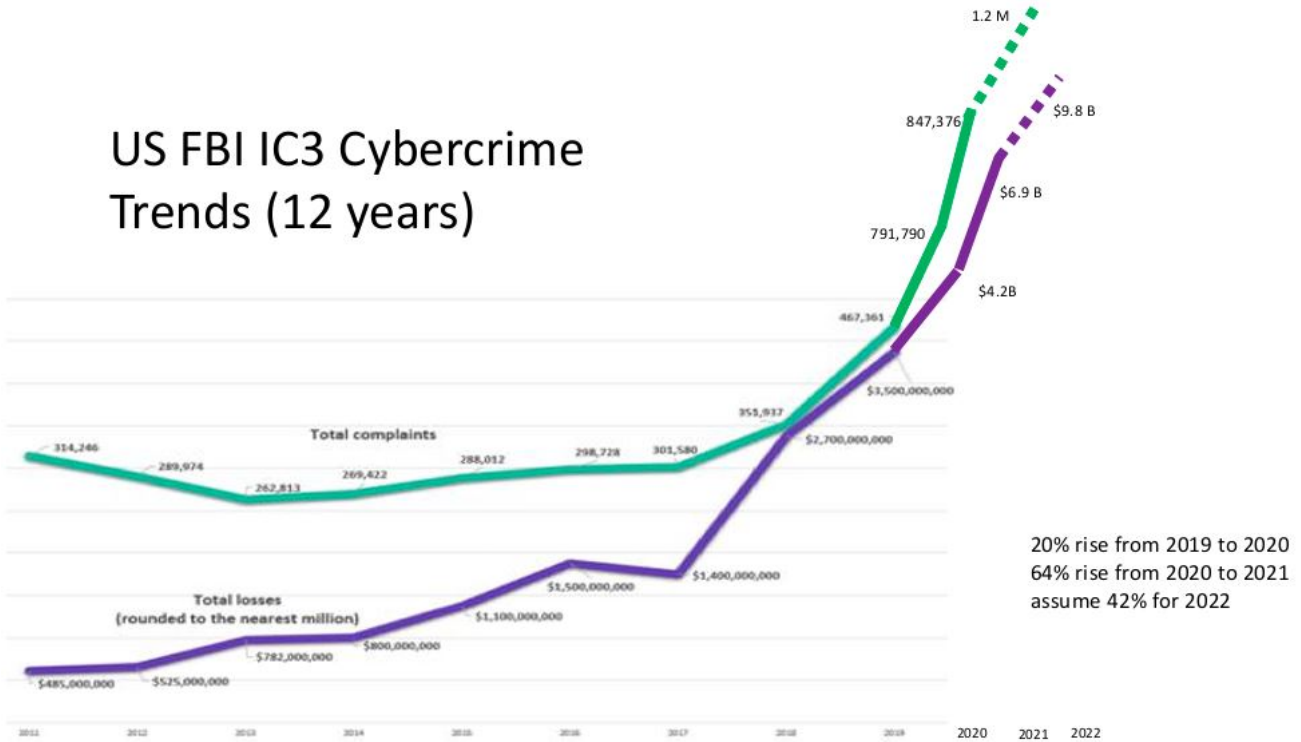
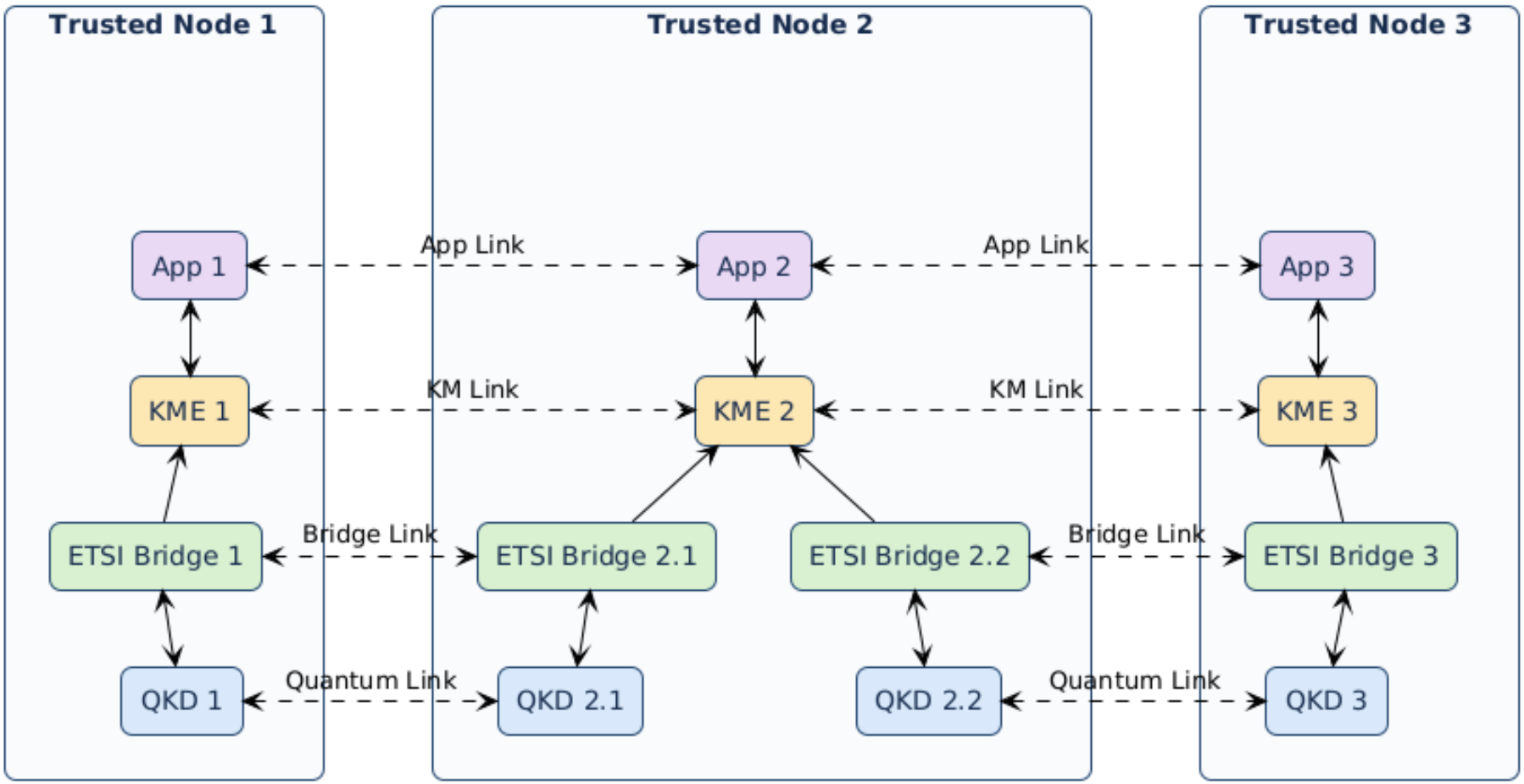


Abb.:
https://www.it-cisq.org/wp-content/uploads/sites/6/2022/11/CPSQ-Report-Nov-22-2.pdf

Beispielhaftes QKD-Netzwerk

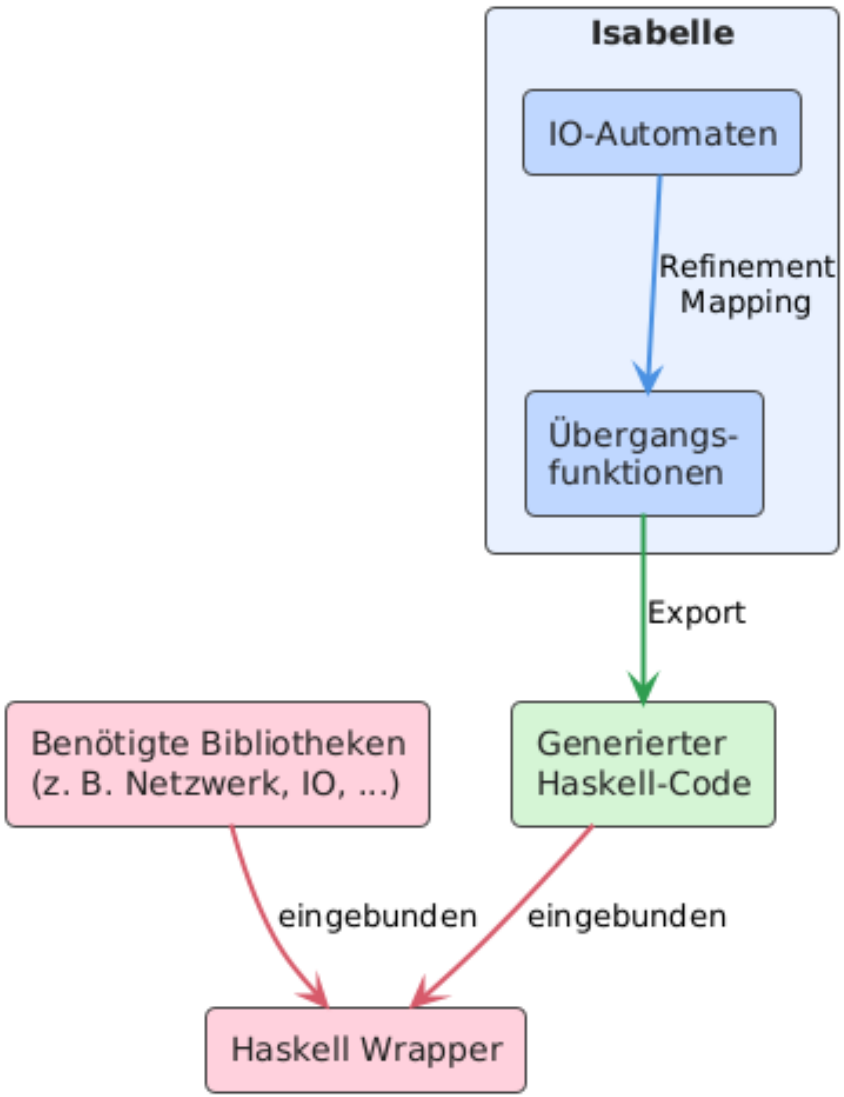


Konsistenz der Schlüssel

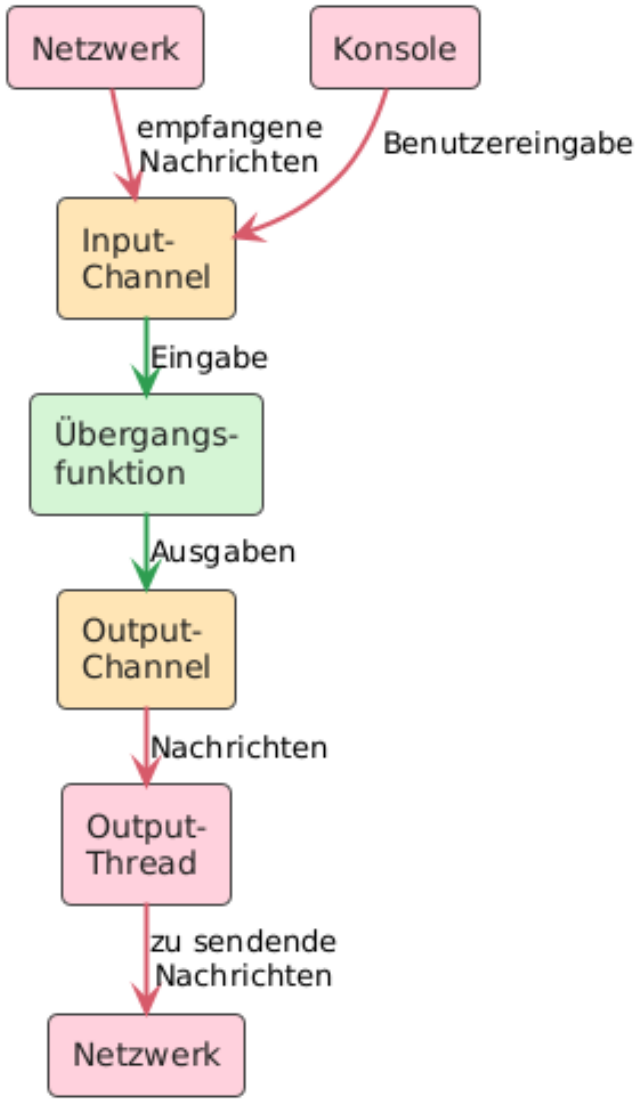
Für alle Anwendungen App_1 und App_2 gilt:

$$\big(\mathrm{App}_1 \text{ besitzt } k_1 \wedge \mathrm{App}_2 \text{ besitzt } k_2 \wedge k_1.\mathrm{id} = k_2.\mathrm{id}\big) \implies k_1.\mathrm{value} = k_2.\mathrm{value}.$$

Von IO-Automaten zum ausführbaren Haskell-Programm



Struktur des Haskell-Wrappers



Forschungsziel
Isabelle Ansatz

+ Haskell-Wrapper kann auf notwendiges IO-Mapping beschränkt werden
+ Beweise außerhalb von Isabelle auf ein Minimum reduziert
+ Eigentlicher Code erst nach der formalen Modellierung in Isabelle
− Zusätzlicher Code durch Wrapper
− Abhängigkeit von der Korrektheit des Codegenerators, Isabelle, verwendete Bibliotheken und Wrappers
− Die Codegenerierung nur begrenzt steuerbar; generierte Code muss evtl. nachbearbeitet werden

Forschungsziel

Formale Verifikation und Implementierung des SQuIRRL-Stacks:

1. Formale Modellierung des Stacks in Isabelle (IOA + Übergangsfkt.)
2. Nachweis der Eigenschaften auf Ebene der IO-Automaten
3. Nachweis der Eigenschaftserhaltung zwischen IOA und Übergangsfkt.
4. Generierung einer Implementierung aus dem formalen Übergangsfunktionen
5. Ausführbarer Haskell-Wrapper für jedes Modul

Quellen

- https://www.dlr.de/de/ar/themen-missionen/raumfahrttechnologien/traegersysteme/ariane-5 (3.9.26)
- https://testomat.io/blog/software-bug-cost/ (3.9.26)
- https://en.wikipedia.org/wiki/Therac-25 (3.9.26)
- https://osm.hpi.de/squirrl/ (8.9.26)
- https://www.it-cisq.org/wp-content/uploads/sites/6/2022/11/CPSQ-Report-Nov-22-2.pdf(8.9.26)